

ÇEPER

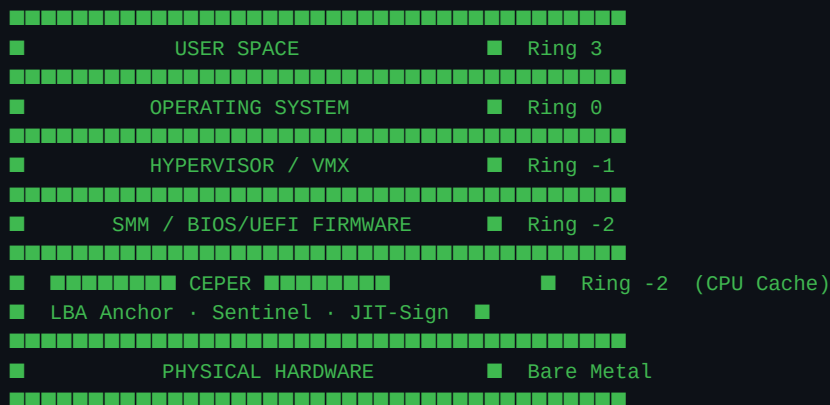
Final Technical Architecture Summary

A Ring -2 Hardware Security Layer

Status: Theoretical Framework · Not Implemented · v0.1-draft

1. Placement and Isolation (Ring -2)

Çeper is a "mechanical" security layer that lives between the operating system and the hardware, residing in the processor cache at Ring -2, protecting itself via physical addressing.



Working Area: Runs directly on the CPU Cache (L1/L2/L3), not RAM (DRAM). Unaffected by any RAM-based attack — DMA, Cold Boot, RowHammer, etc.

Static Structure: On every cold boot, Çeper is loaded from its sealed physical address on disk; when the system shuts down, it is wiped from memory.

```
; Çeper – Cold Boot Load (Pseudocode)
cold_boot_init:
    IN    al, BIOS_CTRL_PORT      ; read signal from BIOS line
    CMP   al, ÇEPER_WAKE_SIGNAL   ; expected signal?
    JNE   halt                   ; no -> stay dormant
    MOV   esi, LBA_ANCHOR_ADDR    ; load fixed LBA address
    CALL  load_to_cache           ; load into Cache, not DRAM
    CALL  verify_hw_signature     ; verify hardware signature
    JMP   sentinel_loop          ; enter sentinel loop
halt:
    HLT
```

2. Physical Address Anchor (LBA Anchor)

Positioning: Çeper locates itself not through the file system but via the hardware-defined LBA (Logical Block Address).

Portability Behaviour: When the disk is attached to another machine, that machine's BIOS does not recognise this specific LBA anchor and does not invoke Çeper — so Çeper never wakes up. The disk remains passive: its data is fully readable and the disk can be wiped on the second machine for a clean reinstall (update cycle).

Self-Defence: While running, Çeper rejects all write and delete requests targeting its own LBA sectors at the hardware level.

```
# LBA Anchor — Conceptual Representation
LBA_ANCHOR = 0x0000_0022      # fixed by manufacturer
SECTOR_COUNT = 8              # 4 KB (8 x 512 B sectors)

def is_write_allowed(lba_target: int, source: str) -> bool:
    if lba_target in range(LBA_ANCHOR, LBA_ANCHOR + SECTOR_COUNT):
        return False          # own sectors: reject at hardware level
    if source != 'TRUSTED_BIOS':
        return False          # source not BIOS: reject
    return True
```

3. Authority and Verification

Source-Oriented Control: Çeper only processes requests arriving from the trusted BIOS/UEFI hardware line. Every command whose source is not the BIOS — OS, drivers, etc. — is immediately discarded.

Anti-Backflashing & Signature Check: During an update, Çeper does not only verify the manufacturer signature — it also checks the version number currently stored in hardware. This prevents rollback attacks where an older, vulnerable firmware is hidden behind a valid signature.

Dynamic Signature (JIT): No signature library is kept on disk. Çeper compares the live hardware signature captured at that moment against the incoming package.

```
TRUSTED_SOURCE = 'BIOS_HARDWARE_LINE'

def dispatch(command, source):
    if source != TRUSTED_SOURCE:
        destroy(command)      # silently discarded, no log
        return
    execute(command)

def validate_update(pkg):
    live_sig = read_hardware_signature() # captured at this instant
    live_ver = read_hardware_version()

    if pkg['signature'] != live_sig: raise SignatureMismatch
    if pkg['version'] <= live_ver: raise RollbackAttempt
    flash(pkg)
```

4. Hardware Sentinel

Physical Limit Lock: Intercepts voltage or over-limit signals heading to the VRM, EC, and battery controller — e.g. Plundervolt — before they reach the hardware.

IOMMU Isolation: Confines peripherals to their own cells, blocking lateral leakage at the physical level.

```
VOLTAGE_CEILING = 1.35          # Volt -- safe upper bound

def sentinel_loop():
    while True:
        v = read_vrm_voltage()
        if v > VOLTAGE_CEILING:    # Plundervolt et al.
            cut_vrm_signal()       # cut before it reaches hardware
            enforce_iommu_isolation() # peripherals locked in cells
```

5. Installation and Update (Wipe & Reload)

Manual Installation: No internet access is involved; Çeper is installed and configured manually.

Clean Cycle: When an update is needed, the disk is wiped on a second machine — where Çeper remains dormant — reattached to the primary machine, and the new Çeper version is written manually to its LBA address via a BIOS tool.

[Initial Installation]

```
Manufacturer -> writes Ceper to LBA Anchor
              -> registers wake signal in BIOS
              -> sealed
```

[Update Cycle]

1. Remove disk -> attach to second machine (Ceper dormant, data readable)
2. Wipe disk on second machine (clean reinstall)
3. Write new Ceper to LBA address (BIOS tool, manual)
4. Reattach -> cold boot -> new version active

6. Why It Remains a Theoretical Framework

Çeper is a technically coherent and implementable architecture. There are two practical reasons it has not been built:

Hardware Constraint

Low-level hardware work of this kind requires a spare machine that can be experimented on — and, at some point, permanently damaged. Testing Çeper requires opening a custom signal channel in the BIOS, writing directly to LBA sectors, and probing voltage limits at their boundaries. Every one of these steps carries a risk of irreversible hardware damage.

Financial Constraint

The only machine available is not one that can be risked for this project. The capital needed to acquire a dedicated test machine does not exist. Keeping the project as a theoretical framework is not a failure — it is a deliberate choice: rather than writing prototypes that cannot be safely tested, the architecture is preserved until a safe environment exists to test it properly.

*Çeper — v0.1-draft | Theoretical Architecture Document
All code blocks are conceptual illustrations; they are not executable production code.*